

CIS 5523 Final Project Report

Using Text Mining to Classify Congressional Bills

Author: Daniel Blitshtein

Email Address: tur43299@temple.edu

Abstract

This research project investigates how text mining can be used to classify congressional bills. As a way of helping Americans understand bills currently being passed. To be more specific, using a feature of text mining known as topic modelling. When it came to the bill I chose to investigate, I choose the “Big Beautifull Bill” (BBB) passed by the current 119th US congress. This also involved creating my own data set and .csv files to implement topic modelling techniques. In this project I learned about topic modelling and made use of two main methods being: Latent Dirichlet Allocation (LDA) which is used to identify global thematic structures, and KeyBERT that extracts semantically meaningful keywords from individual sections. Then I compared & contrasted both techniques and their effectiveness. Ultimately, the results showed that LDA was very successful in describing the bill highlighting policy topics such as taxation and healthcare. Meanwhile, KeyBERT did provide information on the BBB being finically focused policy wise with keywords centered on taxation. However, it had shortcomings by excluding health care keywords and in easily making stop words to filter legal language out compared to LDA. All of this shows that text mining can be useful in understanding congressional bills, but the right methods need to be applied.

Introduction

How many people truly understand the content of congressional bills? Beyond media headlines and bill titles, the average U.S. citizen has limited access to or understanding of the detailed topics and inner workings of legislation. Given the length, complexity, and legal language used in these documents, manually interpreting bills is difficult. This raises an important question: how can we use methods like text mining and its techniques such as topic modelling, to solve this information problem?

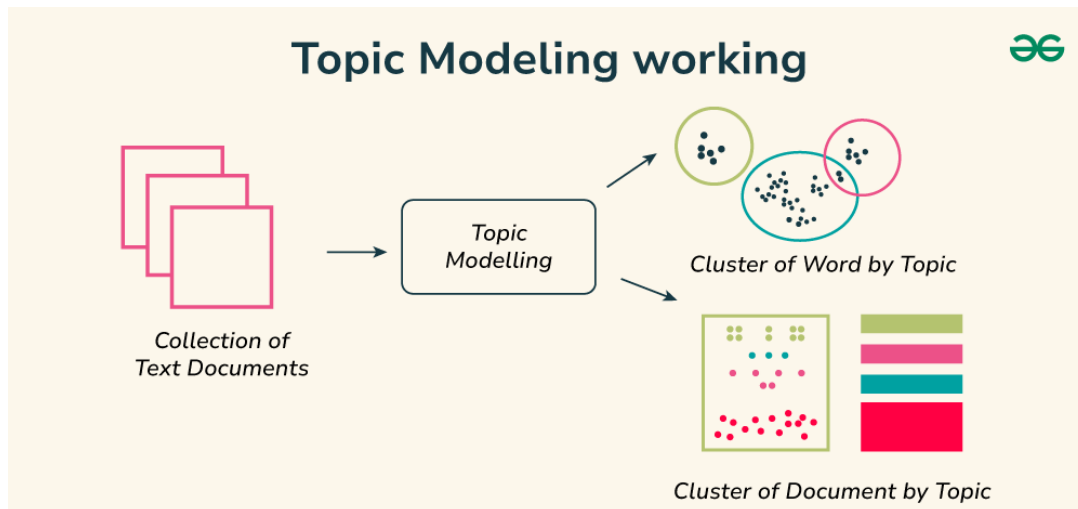


Figure 1: A Diagram of Basic Topic Modelling From [7]

Therefore, my hypothesis is text mining techniques. To be more specific topic modeling, can be used to classify the main topics within congressional bills. For some context, topic modelling uses unsupervised learning in its many algorithms. As shown in figure 1 above, words and documents can be clustered into topic groups [7]. By applying these topic modeling methods, it will allow us to understand what bills represent in an easy to digest format like Taxation, Defense, or Health. Furthermore, my 2 main objectives were the following:

- 1) Determine which topics are the most relevant (shown by most occurrences) for the observed bill/bills.
- 2) Compare and Contrast Different Methods of Topic Modelling on a chosen bill to determine effectiveness at classifying the main topics/keywords.

Related Work

During my research I investigated how others were using text mining to handle government documents and bills. Since, even though I chose to make my own data set it gave me informative knowledge on how other approached the problem. After all, text mining congressional data is a popular topic. It's just not as published in the media right now as AI and LLMs collecting user data to train off.

Anyways, the first thing I found was a very strong tool & dataset python package. More specifically, the pb-congress-scraper Python package. This package provides a framework for collecting and organizing legislative data from public sources, enabling researchers to build structured datasets for downstream analysis [8]. Such tools are essential for transforming raw legislative documents into formats suitable for computational methods, including text mining and natural language processing. This was something I think would have been extremely helpful to

use in my project upon looking at it. Considering I went through the more arduous process of splitting my chosen bill myself, which I touch upon in my methodology section.

Meanwhile, the other important related work I found was a guide on how to approach text mining for government data. The University of Chicago Library provides guidance on working with government documents for text mining, highlighting challenges such as inconsistent formatting, metadata extraction, and the need for preprocessing steps like tokenization and stop word removal [9]. These are some of the challenges I faced when doing my own project and I did incorporate some of the solutions through my own work like the usage of legal stop words.

Now, for the Topic modelling methods I choose to use, topic modeling and keyword extraction are very common approaches to text mining. Latent Dirichlet Allocation (LDA) is widely used to uncover latent thematic structures across collections of documents. Meanwhile, In contrast, keyword extraction methods, such as KeyBERT, focus more on identifying representative phrases within individual documents using semantic(cosine) similarity. In my project I choose to apply both topic modeling methods of LDA and keyword extraction to a large congressional bill. By comparing these methods, I aimed to evaluate their effectiveness in handling the specific challenges posed by legislative text, including the presence of legal boilerplate language and the need for interpretable outputs.

Methodology

My methodology can be described as 5 key sections: Bill Selection & Data Collection, Data Preprocessing, Implementing Topic Modelling with LDA, Implementing Keyword Extraction with KeyBERT, and lasty Comparing-Contrasting both Models and their results.

First, when it comes to selecting a bill to make a data set out and then analyze. I realized immediately that it would be extremely difficult to go through 100+ separate bills and combine them into a data set. So, I pivoted and chose one large bill known as the Big Beautifull Bill. A bill was passed by the current 119th Congress and made headlines last year. That I obtained through [1][2] and got access to the direct pdf [3] of H.R.1(BBB) bill. The bill was 1018 pages total and is what's considered in politics to be a mega bill. That is a bill with multiple policy objectives combined. Thus, making it perfect to make a data set out to then analyze. In Figures 2 & 3 below you can see what text in the bill documentation looks like.

119TH CONGRESS
1ST SESSION

H. R. 1

AN ACT

To provide for reconciliation pursuant to title II of H. Con.
Res. 14.

1 *Be it enacted by the Senate and House of Representa-*
2 *tives of the United States of America in Congress assembled,*

TITLE I—COMMITTEE ON AGRICULTURE

Subtitle A—Nutrition

- Sec. 10001. Thrifty food plan.
- Sec. 10002. Able bodied adults without dependents work requirements.
- Sec. 10003. Able bodied adults without dependents waivers.
- Sec. 10004. Availability of standard utility allowances based on receipt of energy assistance.

Figures 2 & 3: From [3] show how text in the BBB looked like and why I needed to divide it by Title and Section when making my Data Set.

Next, onto the Data preprocessing phase, because the bill was very large. I needed to apply multiple preprocessing steps required before doing any text mining techniques. I First made sure the PDF document [3] was parsed using the pypdf library to extract text content page by page. I then made sure the extracted text was concatenated into a single document and cleaned to remove any broken words, excessive whitespace, and irregular line breaks. That way I could segment the document into meaningful units using regular expressions that identify congressional structural markers. I did so for Titles, Subtitles, Parts, Subparts, and Sections (e.g., “SEC. 10001.”). Basically, each section was treated as an individual unit of analysis. This allowed me to essentially chunk out the entire document into something analyzable.

Then, to improve granularity, I got rid of any extremely small section. I also made sure the long sections were further divided into overlapping text chunks of approximately 800 words, with an overlap of 100 words. That allowed for my chunk sizes to stay consistent when I later ran both of my models to analyze.

```
LEGAL_STOPWORDS = {  
    "act", "section", "subsection", "paragraph", "subparagraph", "clause",  
    "title", "subtitle", "part", "subpart", "secretary", "administrator",  
    "shall", "may", "including", "insert", "strike", "amend", "amended",  
    "relating", "thereof", "herein", "amount", "amounts", "funds", "program"  
}
```

Figure 4: Legal Stop words I used for my LDA Algorithm.

Finally, the text was normalized by converting to lowercase, removing punctuation, and filtering out stop words. I also had to make sure the models covered relevant information such as health care terminology, instead of over reporting legal terminology. So, I made some stop words to remove procedural language common in legislation (E.X: act, section, paragraphs and more). This was very useful for both models, with Figure 4 above showing the words I blocked for LDA. This is something I’ll discuss later.

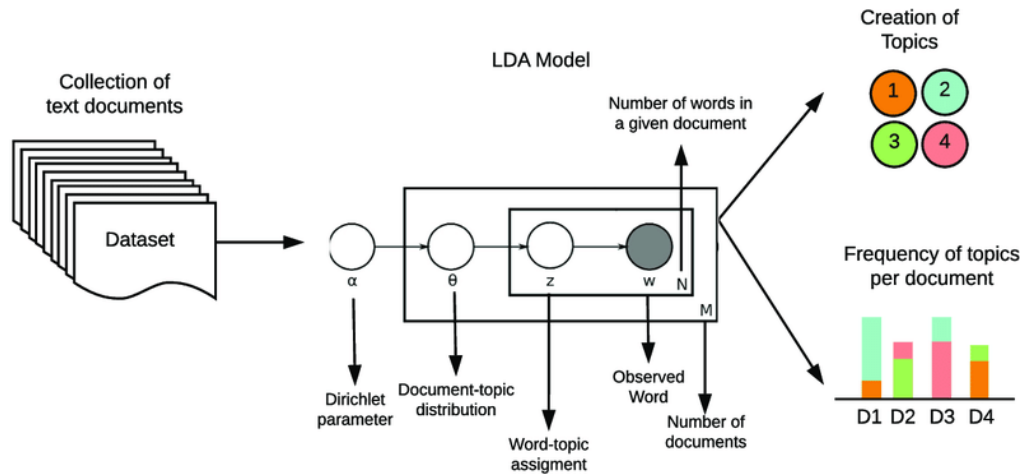


Figure 5: Showing the Architecture of an LDA Model.

Now, onto step 3 is implementing Latent Dirichlet Allocation (LDA). LDA is topic modelling method that uses a generative probabilistic version that assumes each report is a combination of a small number of subjects [4][5], and that every word's creation is because of one of the record's subjects. Basically, LDA finds out what topics exist, and then which words within the documents belong to each topic through clustering, with a depiction of the model seen above in Figure 5.

Going more in-depth, text chunks were vectorized using a bag-of-words representation via Count Vectorizer, with any stop words removed. I trained my LDA model to use ($k = 10$) topics, as I learned from [4] that 10-20 topics are the best to use for these models. I also made sure that one of the topics would include any remaining legal words that weren't blocked out by my stop words.

For each topic, the top contributing words were extracted to facilitate interpretation. Additionally, each text chunk was assigned to a dominant topic based on the highest probability in the topic distribution. This allowed for the analysis of how different sections of the bill align with underlying policy themes.

Meanwhile, the other method I implemented was KeyBERT, depicted below in figure 6.

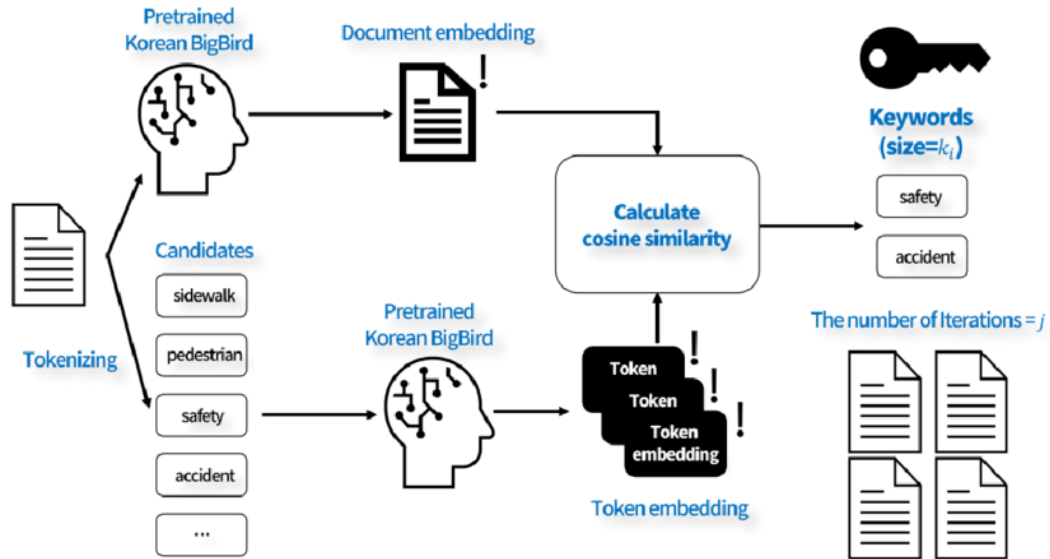


Figure 6: Showing the Architecture of a KeyBERT Model.

KeyBERT, involves keyword, it leverages transformer-based embeddings to identify key phrases that are semantically representative of a document [5][6]. Then the preprocessed text chunks are passed into a KeyBERT model using the “all-MiniLM-L6-v2” embedding model. This also required me to set up my own virtual environment in the kernel to run KeyBERT properly. Keywords were extracted in batch mode to improve computational efficiency. For each chunk, the top five keywords or key phrases were selected based on cosine similarity between the document embedding and candidate phrase embeddings.

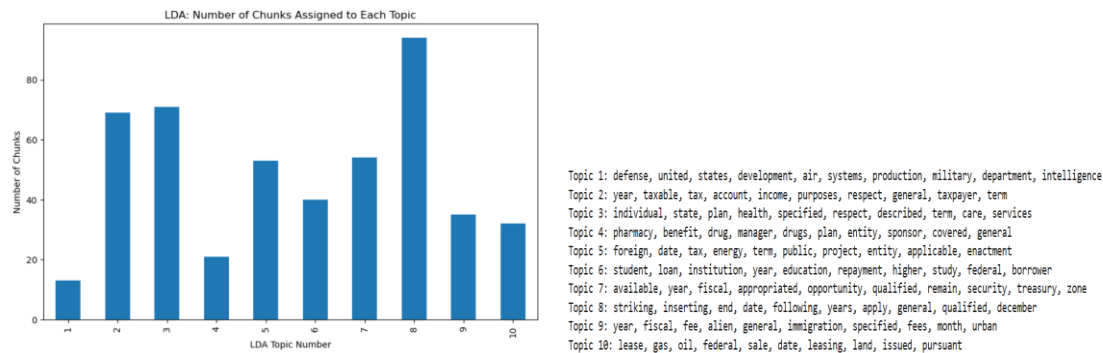
Just like LDA, I needed to apply an additional cleaning step prior to keyword extraction. This involved trying to remove legal stop words that persisted after initial preprocessing. As shown in Figure 7 below. This Step was very important for KeyBERT since it was a lot more sensitive to the legal language than LDA was.

```
LEGAL_STOPWORDS = {
    "act", "section", "subsection", "paragraph", "subparagraph", "clause",
    "title", "subtitle", "part", "subpart",
    "secretary", "administrator",
    "shall", "may", "including", "include",
    "insert", "inserting", "strike", "striking",
    "amend", "amended", "amendment",
    "following", "date", "general", "term", "respect",
    "applicable", "enactment", "years", "year",
    "thereof", "herein", "provided", "provides",
    "means", "definition", "defined"
}
```

Figure 7: Legal Stop words I used for my KeyBERT algorithm

Lastly, I compared the effectiveness and results of LDA and KeyBERT when it came to analyzing the BBB, and what makes them different from each other. Along with which model I believe would be more effective for analyzing congressional bills in my discussion section.

Results



Figures 8 & 9: On the left is a Bar Chart showing the Number of Chunks assigned when I ran LDA. While on the right is the corresponding topic words assigned to each topic by the LDA algorithm.

Onto my results, as shown above in Figures 8 & 9. I was able to have the LDA create 10 topics(with associated terms) and plot out the number of chunks(parts of the bill) they occurred. These 10 topics can be summarized as: Military-Defense, Tax-Policy, HealthCare, Drug Policy, Mixed Topics, Education, Budgeting, Legal Language(existed to separate out legal language from actual policy), Immigration, and lasty Energy.

Overall, I noticed that Topics 5(Mixed Topics) & 7(Budgeting) we're a bit more mixed with terms for the topic generation with some overlap, which I believed was due to have 10 topics causing a higher chance of overlapping terms. On the other hand, Topics 2(Tax Policy) & 3(HealthCare) had the highest amount of chunks matching to them which makes sense for a Mega Bill like the BBB that was mainly rooted in Tax Policy, and Healthcare(Medicaid Cuts, etc.). Another usefull element was Topic 8(Legal Language) which served as a way of pooling together any legal language I forgot to block out in my data preprocessing. This shows how LDA can be used to effectively show what topics are most occuring and in turn allow us to understand what the BBB represents.

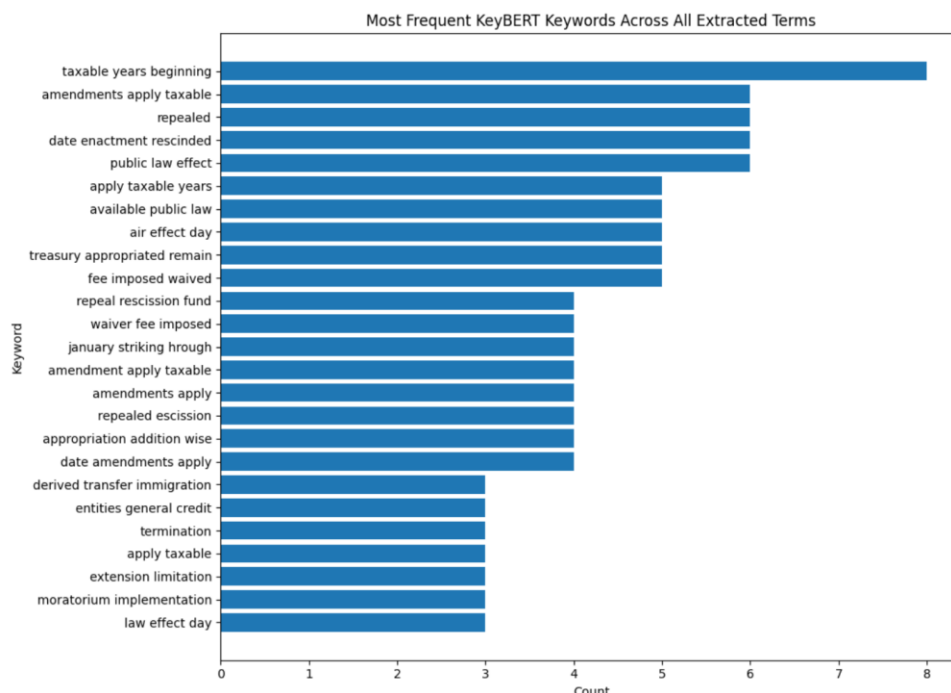


Figure 10: A Bar graph showing the most frequent keywords using KeyBERT across the extracted terms, as well as the top keywords and their counts.

Meanwhile, for my KeyBERT results, as shown in Figure 9 above. I was able to run KeyBERT successfully, but it was extremely difficult to manage the keywords, something I'll touch on more in the discussion. However, I was still able to infer what the BBB represented through these results, as being a Taxation focused bill. Where "taxable year beginnings", "apply taxable years" and other keywords showed the bill to me focused on taxation. The big weakness I observed was that Health care terms were less prevalent than expected compared to the LDA. Where only "repealed" as a common term could be used to infer that information. Still, I was able to infer the general idea of what the bill represented for taxation using the KeyBERT results. Although, that doesn't mean that it's a perfect method as I'll discuss below.

Discussion

Throughout this research project. I was able to learn not only how to make my own data sets out of the congressional data. But also apply the Latent Dirichlet Allocation and KeyBERT models successfully.

The most effective model in my view was LDA. It proved to be highly effective in identifying the global thematic structure of the bill. Since it was able to successfully group words into coherent topics that aligned with real policy areas, such as taxation, healthcare, defense, immigration, and energy. While also making a separate topic to store and legal terms I missed with my stop words. To me this shows that LDA is well prepared to be applied to other

congressional bill documents. Something I wish I had the time to do if I had more time during the project. By assigning each text chunk a dominant topic, LDA also provided insight into how different sections of the bill relate to one another at a structural level. However, one limitation of the model is that by reserving a topic for legal language it means that maybe more topics will be required than just 10. Or that I'd need to apply so many stop words that it can negatively impact the dataset.

Meanwhile, KeyBERT focused on extracting local, section-level keyphrases using semantic embeddings. This approach produced more interpretable outputs in the form of short phrases, making it easier to interpret. For instance, "repealed", "taxable years beginning" are clear keywords. Although, one of the issues that came with this model is that Keywords are very sensitive, especially to any legal language that I didn't account for with the many stop words I added. I needed to add a lot more words compared to LDA. To get results that allowed me to interpret the bill as a taxation-based bill. Even then, KeyBERT still proved to be more difficult at inferring the bills contents than LDA when comparing the two models.

This is partly caused by one key difference between the two methods, which is their level of abstraction. LDA operates at a higher level, identifying latent topics across the entire corpus [4], whereas KeyBERT operates at a lower level, focusing on the most representative phrases within individual text chunks [5]. As a result, LDA provides a more comprehensive overview of the bill's policy landscape, while KeyBERT is more of a localized approach.

So, for the problem of using a topic modelling technique on a congressional bill LDA is the logical choice. Since we want to look at things from that higher perspective to then interpret those results in an easily understandable way. Therefore, LDA is more effective for understanding the overall structure and major themes of a large legislative document, making it useful for summarization at scale. KeyBERT, on the other hand, is more useful for generating readable summaries of specific sections after some fine tuning.

On top of that, KeyBERT was very computationally expensive due to its reliance on transformer-based models, I recall having to wait 20+ minutes for KeyBERT to finish running its model since it needed to connect to Hugging Face and the transformer libraries. I also had to modify my kernel into a virtual environment that allowed me to use KeyBERT in Jupyter notebooks that took some time.

Overall, the results support my hypothesis that text mining; to be more specific topic modelling techniques can help us understand and classify the content of congressional bills. However, the choice of method plays a critical role in the quality of the output. Where topic modelers like LDA provide stronger global insights, while KeyBERT offers better interpretability at the local level. Any Future projects could explore potentially finding a way to merge both methods. Another thing that should be looked at are the many other topic modelling techniques such as: Dynamic Topic Models (DTM), and Corelated Topic Models(CTM), and LSA(Latent Semantic

Analysis). I also believe that LLM's have a lot of potential in being able to summarize or give us understanding of congressional bills. So that may also be another avenue of exploration.

Acknowledgments

Overall, this research project has allowed me to become more informed about text mining and topic modelling techniques. In contrast to some of the other Grad class final projects I've had as a +1 student this project allowed for a lot more freedom & helpful feedback during the learning process.

I'd also like to give some thanks to Shelly. Who put me on the Idea of looking into KeyBERT as a modelling technique. Since it approaches topic modeling from the keyword perspective rather than grouping by topics it provided a great contrasting method to LDA. Even though it wasn't as successful in interpreting the full scope of a congressional bill. I still learned a lot about how KeyBERT works and how it can be applied to text mining problems. Finally, the Supplemental materials for this project including the code, csv files, and more. That I made during the data preprocessing stage are included in the GitHub link here: <https://github.com/Curb02/CIS-5523-Research-Project>.

References

- [1] U.S. Congress, "Congress.gov," *Congress.gov*. Available: <https://www.congress.gov/>.
- [2] U.S. Government Publishing Office, "Congressional Documents Collection," *GovInfo*. Available: <https://www.govinfo.gov/app/collection/cdoc/119-2025-2026/hdoc/%5b100%20-%20199>.
- [3] U.S. Congress, "H.R.1 — Bill Text (119th Congress)," *Congress.gov*. Available: <https://www.congress.gov/119/bills/hr1/BILLS-119hr1eh.pdf>.
- [4] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet Allocation," *Journal of Machine Learning Research*, vol. 3, pp. 993–1022, 2003.
- [5] GeeksforGeeks, "Text Mining in Data Mining," *GeeksforGeeks*. Available: <https://www.geeksforgeeks.org/nlp/text-mining-in-data-mining/>. Accessed: Apr. 28, 2026.
- [6] PyPI, "KeyBERT," *Python Package Index*. Available: <https://pypi.org/project/keybert/>.
- [7] GeeksforGeeks, "What is Topic Modeling," *GeeksforGeeks*. Available: <https://www.geeksforgeeks.org/nlp/what-is-topic-modeling/>.
- [8] PyPI, "pb-congress-scraper," *Python Package Index*. Available: <https://pypi.org/project/pb-congress-scraper/>.
- [9] University of Chicago Library, "Government Documents for Text Mining," *UChicago Library Guides*. Available: <https://guides.lib.uchicago.edu/textmining/govdocs>.